# Introduction to Leonardo Supercomputer


**Caterina Caravita**
HPC Production and Support Specialist

c.caravita@cineca.it

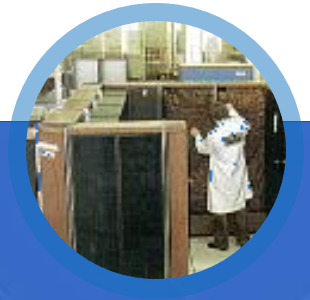
February 18th, 2026

# 50+ YEARS OF SUPERCOMPUTING

## TIMELINE OF CINECA HPC SYSTEMS

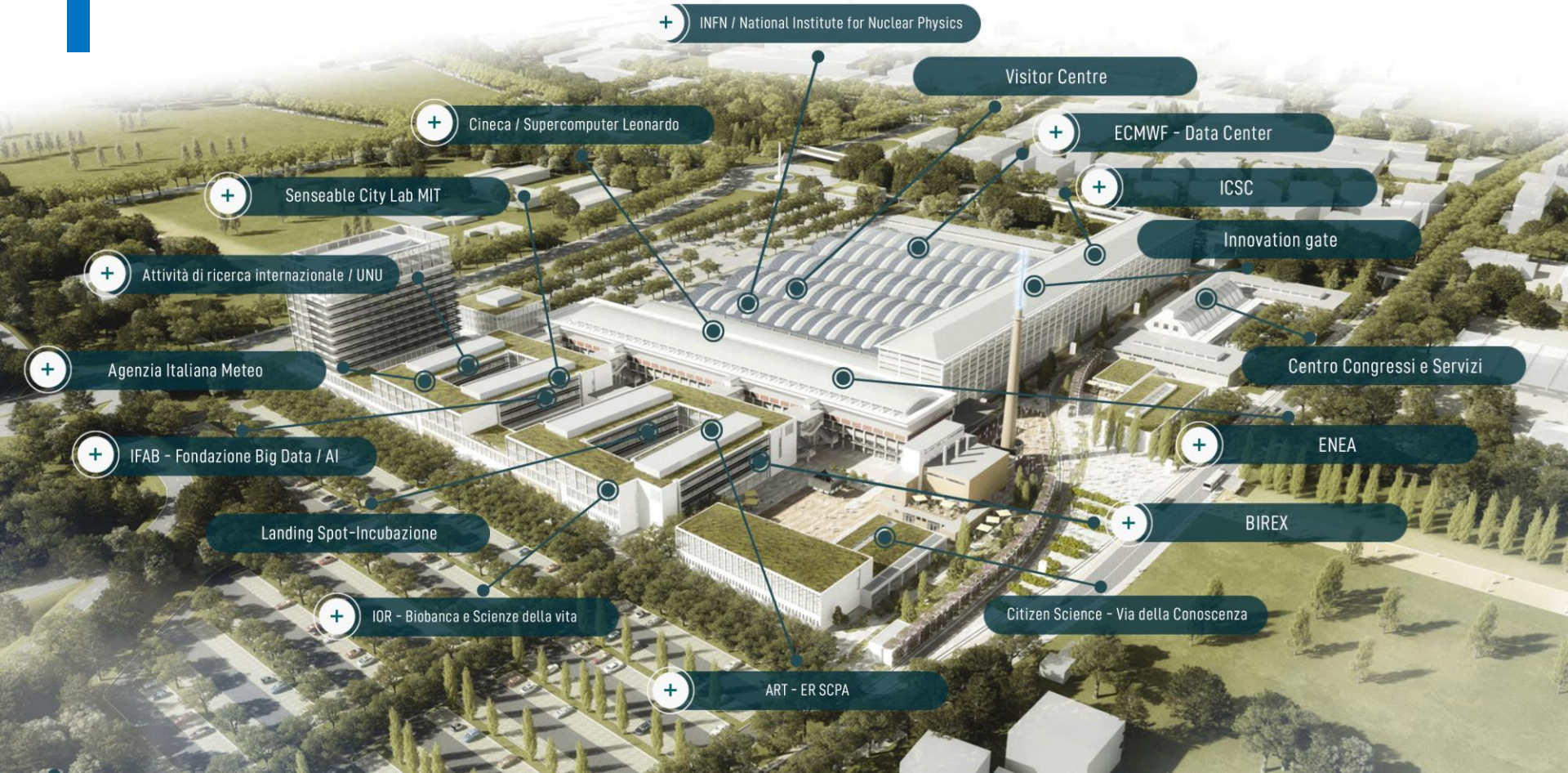
| 1969 | 1975 | 1985 | 1989 | 1993 | 1994 | 1995 | 1996 | 1998 | 199 |
|---|---|---|---|---|---|---|---|---|---|
| CDC 6600 | CDC 7600 | CRAY X-MP / 48 | CRAY Y-MP / 48<br>IBM 3090-600 VF | CRAY C-90/2128 | CRAY T30 64 | CRAY T3D 128 | CRAY T3D 128 | CRAY T3E 256<br>SGI ORIGIN 16 | CRAY T3E 256<br>SGI ORIGIN 64<br>IBM SP POWER 3 |
| 1st Supercomputer in Italy | | | | | 1° PARALLEL SYSTEM | | | | |

| 2000 | 2002 | 2004 | 2005 | 2007 | 2009 | 2012 | 2016 | 2020 | 2023 |
|---|---|---|---|---|---|---|---|---|---|
| | 1 TFLOPS | | | 10 TFLOPS | 100 TFLOPS | 2 PFLOPS | | 32 PFLOPS<br>(first with GPUs) | 250PFLOPS<br>(pre-exascale) |
| VIRTUAL THEATRE | IBM SP4 612 | IBM SP5 512 | IBM CLX<br>XEON<br>1024 | IBM BCX<br>AMD<br>5120 | IBM SP6 | FERMI | MARCONI | MARCONI100 | LEONARDO |

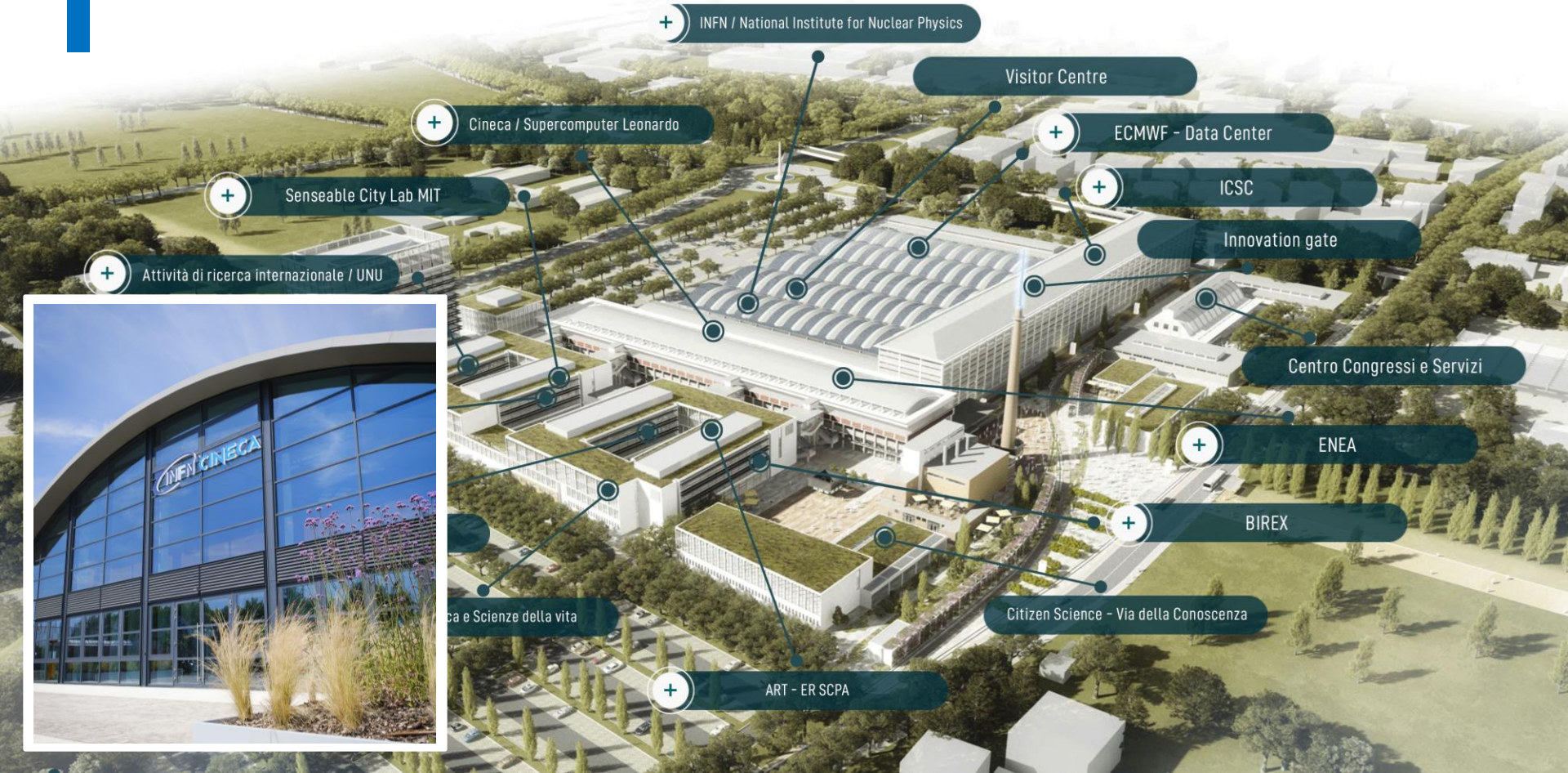


2025
PITAGORA
for Eurofusion

# TECHNOPOLE PARTNERS



# TECHNOPOLE PARTNERS

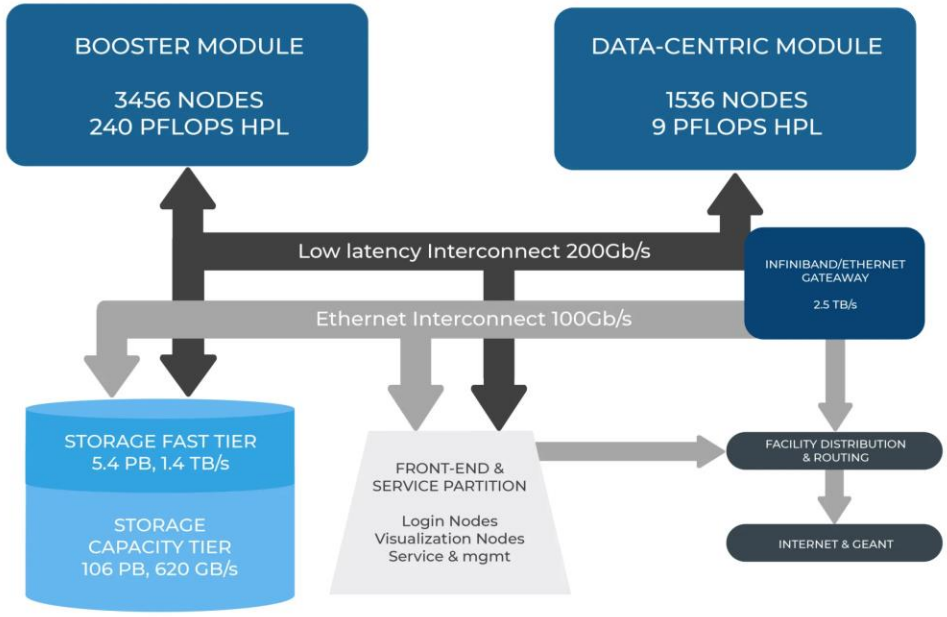


# TECHNOPOLE PARTNERS

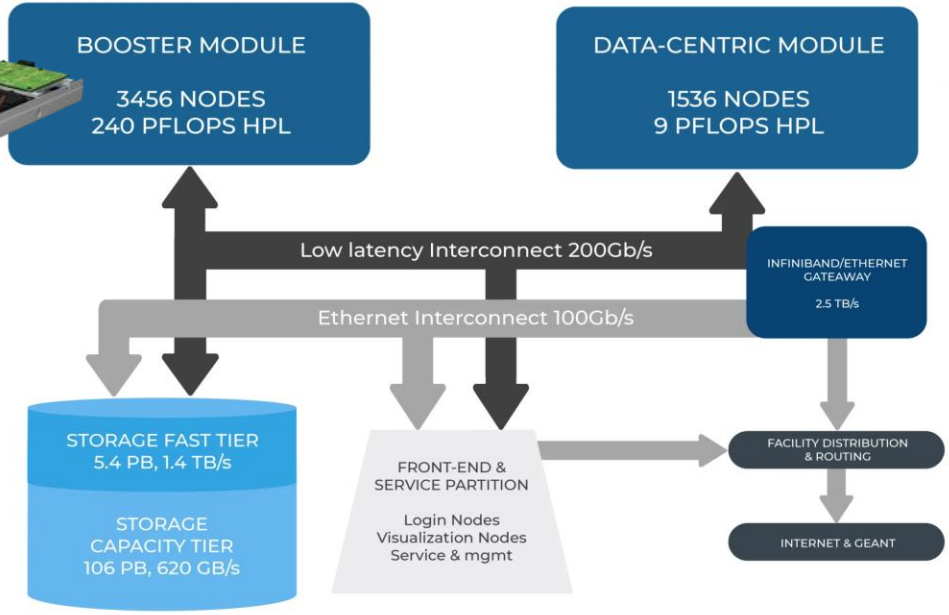
# OUTLINE

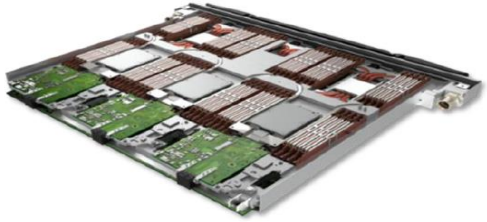
- **Leonardo infrastructure**
- Access HPC resources and filesystems
- Software environment
- Programming environment
- Production environment
- Final remarks

# LEONARDO INFRASTRUCTURE

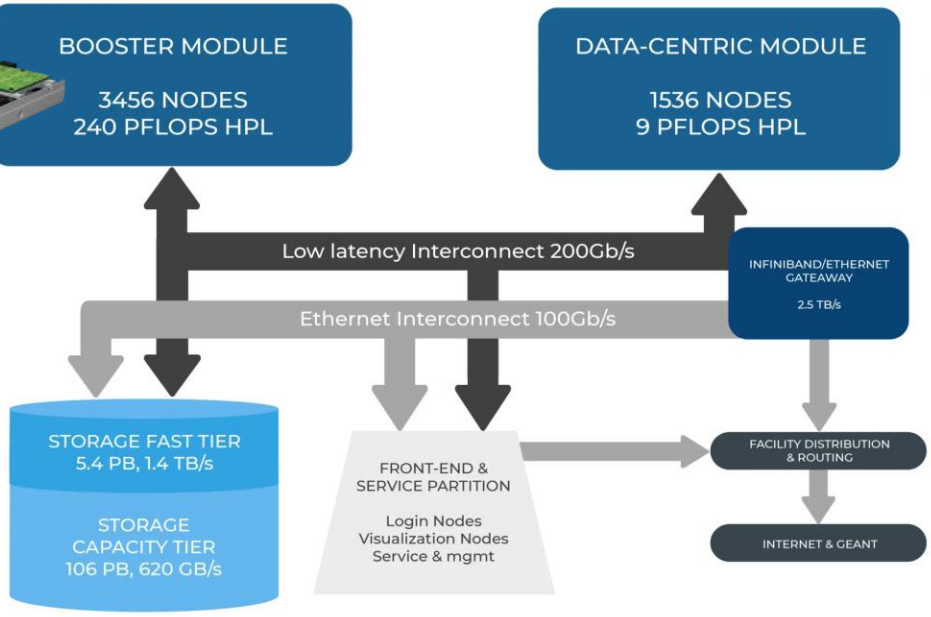


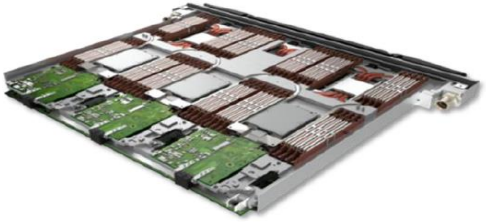

# LEONARDO INFRASTRUCTURE

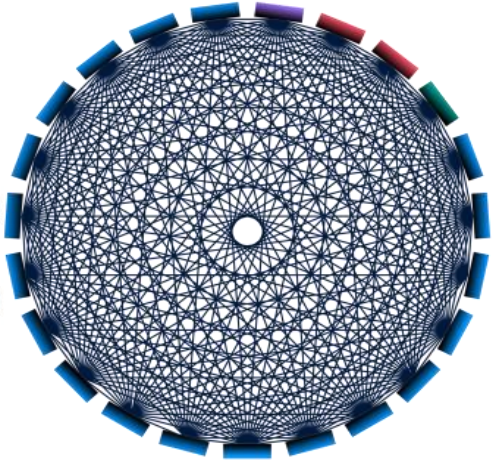


# LEONARDO INFRASTRUCTURE





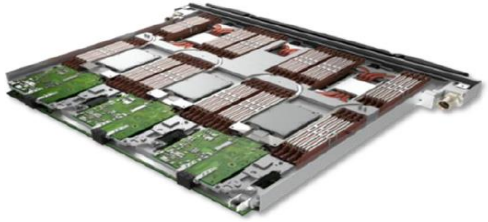
# LEONARDO INFRASTRUCTURE

BOOSTER MODULE

3456 NODES
240 PFLOPS HPL

DATA-CENTRIC MODULE

1536 NODES
9 PFLOPS HPL



Low latency Interconnect 200Gb/s

Ethernet Interconnect 100Gb/s

INFINIBAND/ETHERNET GATEAWAY

2.5 TB/s



STORAGE FAST TIER
5.4 PB, 1.4 TB/s

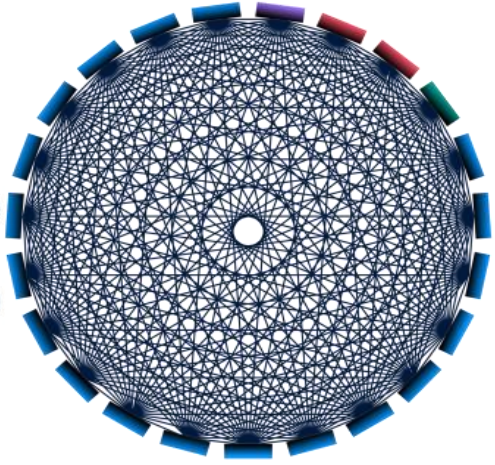
STORAGE
CAPACITY TIER
106 PB, 620 GB/s

FRONT-END &
SERVICE PARTITION

Login Nodes
Visualization Nodes
Service & mgmt

FACILITY DISTRIBUTION & ROUTING

INTERNET & GEANT



# TOP500
## NOVEMBER 2025

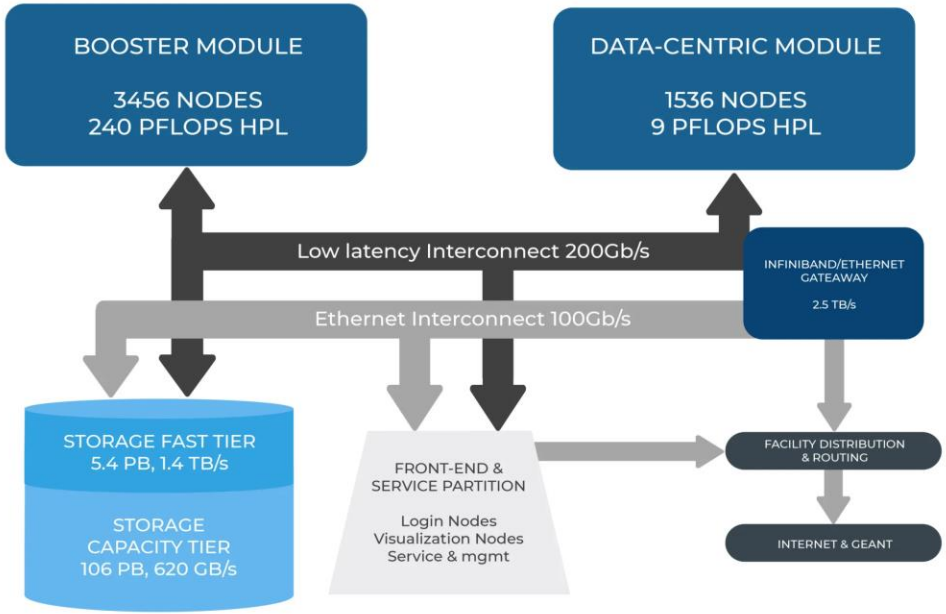
TOP500 The List.

***4th – November 2022***
***10th – November 2025***

After three years of production...
Leonardo Booster is still in the **Top10** of the most powerful supercomputers in the world.

| Rank | System | Cores | Rmax (PFlop/s) | Rpeak (PFlop/s) | Power (kW) |
|---|---|---|---|---|---|
| 1 | **El Capitan** - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, **HPE**<br>DOE/NNSA/LLNL<br>United States | 11,340,000 | 1,809.00 | 2,821.10 | 29,685 |
| 2 | **Frontier** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, **HPE**<br>DOE/SC/Oak Ridge National Laboratory<br>United States | 9,066,176 | 1,353.00 | 2,055.72 | 24,607 |
| 3 | **Aurora** - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, **Intel**<br>DOE/SC/Argonne National Laboratory<br>United States | 9,264,128 | 1,012.00 | 1,980.01 | 38,698 |
| 4 | **JUPITER Booster** - BullSequana XH3000, GH Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, RedHat Enterprise Linux, **EVIDEN**<br>EuroHPC/FZJ<br>Germany | 4,801,344 | 1,000.00 | 1,226.28 | 15,794 |
| 5 | **Eagle** - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR, **Microsoft Azure**<br>Microsoft Azure<br>United States | 2,073,600 | 561.20 | 846.84 | |
| 6 | **HPC6** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, RHEL 8.9, **HPE**<br>Eni S.p.A.<br>Italy | 3,143,520 | 477.90 | 606.97 | 8,461 |
| 7 | **Supercomputer Fugaku** - Supercomputer Fugaku, A64FX 48C 2.2GHz, Tofu interconnect D, **Fujitsu**<br>RIKEN Center for Computational Science<br>Japan | 7,630,848 | 442.01 | 537.21 | 29,899 |
| 8 | **Alps** - HPE Cray EX254n, NVIDIA Grace 72C 3.1GHz, NVIDIA GH200 Superchip, Slingshot-11, HPE Cray OS, **HPE**<br>Swiss National Supercomputing Centre (CSCS)<br>Switzerland | 2,121,600 | 434.90 | 574.84 | 7,124 |
| 9 | **LUMI** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, **HPE**<br>EuroHPC/CSC<br>Finland | 2,752,704 | 379.70 | 531.51 | 7,107 |
| 10 | **Leonardo** - BullSequana XH2000, Xeon Platinum 8358 32C 2.6GHz, NVIDIA A100 SXM4 64 GB, Quad-rail NVIDIA HDR100 Infiniband, **EVIDEN**<br>EuroHPC/CINECA<br>Italy | 1,824,768 | 241.20 | 306.31 | 7,494 |

# LEONARDO ARCHITECTURE




## LOGIN NODES

**2x Intel Xeon Platinum 8358 Processor**
*(Intel Ice Lake – 32 cores, 3.4 GHz with Turbo)*

- Hyper-threading (×2) is enabled
- RAM: 512 GiB RAM DDR4 3200MHz
- 14TiB disk in RAID1 configuration
- No GPUs
- Open to outside network

- ***Serial* partition** on two login nodes
- ***Datamover* service** on three login nodes

# BOOSTER MODULE
## (GPU-accelerated)

### COMPUTE NODES

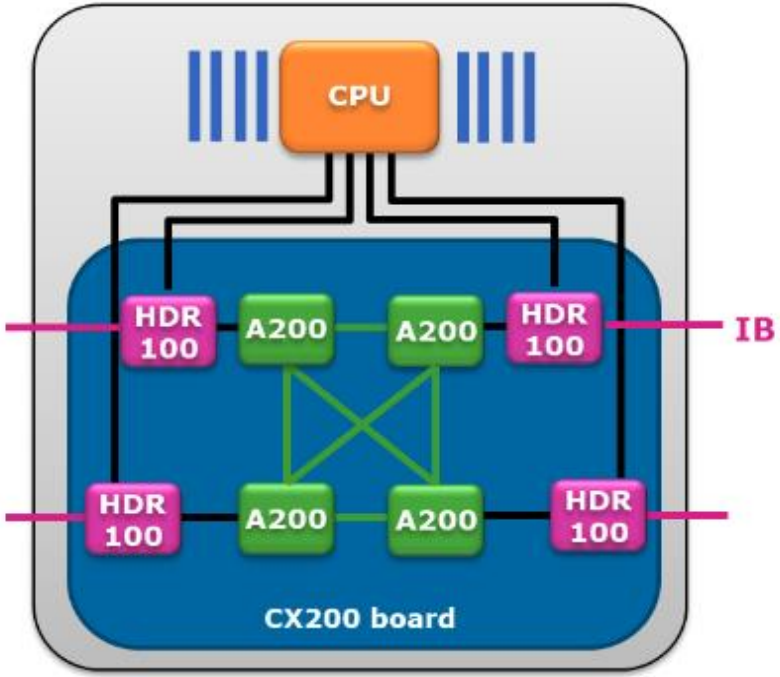
#### Atos BullSequana X2135 "Da Vinci" blade

- 3456 nodes: lrdn[0001-3456]
- 1 × Intel Xeon Platinum 8358 Processor - 32 cores
- RAM: 512 (8 x 64) GB DDR4 3200 MHz
- **Accelerators: 4 × GPU NVIDIA A100 64 GB custom**
  (15% performance improvement over the standard A100)
- Internal network: **NVIDIA Mellanox HDR DragonFly+ 200Gb/s**
- Diskless

**Rmax per node: ~70 TFLOPS**
**Rmax: ~241 PFLOPS**

*Rmax = *Maximal LINPACK performance achieved*

# BOOSTER MODULE
## (GPU-accelerated)




### GPU peak performance

- 11.2 TFlops Peak FP64 per GPU
- 22.4 TFlops Peak FP32 per GPU

### Intra-node connections

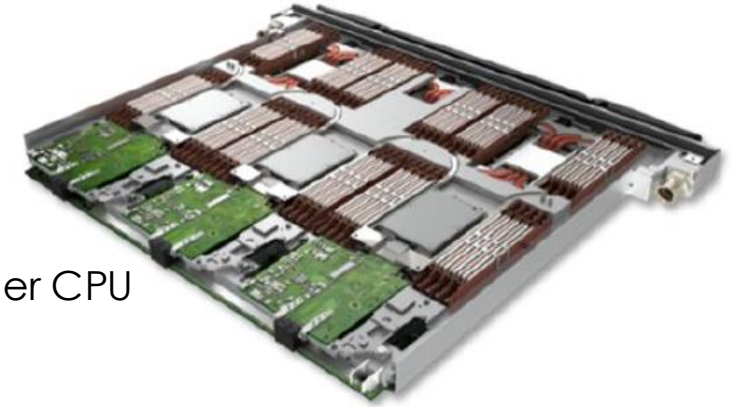
- NVLink, PCIe, GPU direct
- 200 GB/s between the GPU pairs
- Each GPU has direct 100Gb/s connection to the InfiniBand network
- PCIe Gen4 @ 31.5 GB/s

### Memory

- 6.5 TB/s GPU memory bandwidth

# DCGP MODULE
## (CPU-only)

### COMPUTE NODES



#### BullSequana X2140 three-node CPU Blade

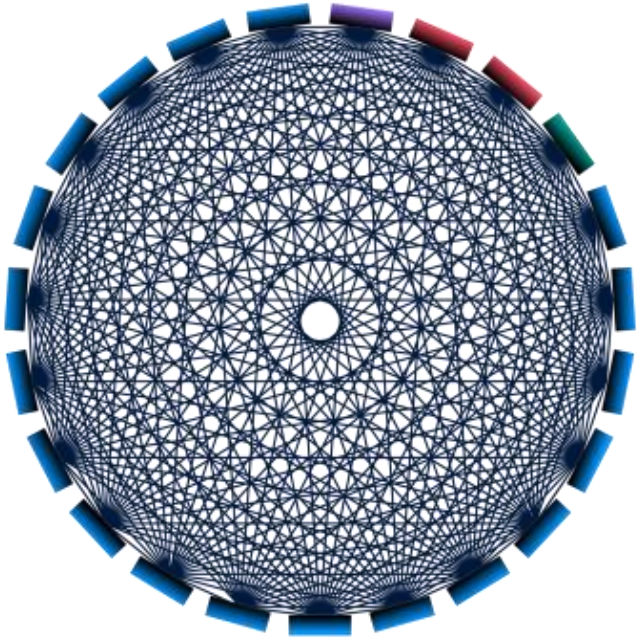
- 1536 nodes: lrdn[3457-4992]
- 2 × Intel Xeon Platinum 8480+ Processor - 56 cores per CPU
- RAM: 512 GB DDR5 4800 MHz
- Infiniband: 1 × NVIDIA HDR cards 100 Gbps via PCIe Gen 5
- Disk: 1 M.2 SSD 3.84 TB

**Rmax per node: ~8.5 TFLOPS**
**Rmax: ~13 PFLOPS**

*Rmax = *Maximal LINPACK performance achieved*

# INTER-NODE NETWORK TOPOLOGY

## shared between Booster and DCGP modules




### Dragonfly+ topology

based on **Nvidia Mellanox Infiniband HDR**, bidirectional bandwidth of 200 Gb/s

- All nodes are divided into cells
- Non-blocking, **two-layer Fat Tree** within the cells
- **All to all** connection between cells

- The scheduler **SLURM** is aware of the network topology and tunes the node allocations
- **Adaptive routing algorithm** alleviates network congestion

# STORAGE

## shared between Booster and DCGP modules

**Fast Tier**
**5.4 PB @ 1.4 TB/s**

**SSD disks (NVMe)**
**(home + public + fast scratch)**


**Capacity Tier**
**106 PB @ read 744 GB/s - write 620 GB/s**

**HDD disks**
**(work + large scratch + DRES)**


- connected via **Infiniband network**
- managed by the parallel filesystem **Lustre**

# OUTLINE

- Leonardo infrastructure
- **Access HPC resources and filesystems**
- Software environment
- Programming environment
- Production environment
- Final remarks

# Access to Leonardo

To **log into Leonardo**, you will need a **HPC username**: you will access via **2 factor authentication (2FA)** through a **login node** and will be redirected to your **$HOME area**.

To **use Leonardo**, you will also need a **Project Account**, associated to your HPC username, which allows you to submit jobs on **compute nodes** through the scheduler **SLURM**.

# Become a new HPC user

- **Register on the UserDB Portal**

  https://userdb.hpc.cineca.it/

- **Get associated to an active Project Account**

  → **Principal Investigator (PI)**: we create the account and set you as PI on the UserDB

  → **Collaborator**: ask your PI to associate you to the account on the UserDB

- **Request the "HPC Access" on UserDB**

  → You will receive soon your credentials by mail

https://docs.hpc.cineca.it/general/users_account.html

# HPC username

The HPC username identifies the individual connecting to the system.

Usernames are **strictly personal**. Every user entitled with login credentials is to be considered personally responsible for any misuse that should take place.

Typically, usernames are:

- 8 characters long
- composed using the first letter of your name and the first 7 letters of your surname
- trailing zeros if your surname is less than 7 letters
- successive numbers in case of already existing username

e.g. **Mario Rossi** -> **mrossi00**

# Access via 2-factor authentication

## First time

- **Activate the 2FA**
  Authenticate on our **Identity Provider** at https://sso.hpc.cineca.it with your **HPC username and password**.
  → You will need an **app to generate authentication codes** (e.g. Google Authenticator).

- **Install and configure the smallstep client**
  The procedure depends on your OS.

https://docs.hpc.cineca.it/general/access.html

# Access via 2-factor authentication

**"Any" access to the cluster**

- **Request the ssh certificate** to our Identity Provider via the smallstep client
  → A web page will open on the **browser** and you will be asked to insert **HPC username and password**, and a **One-Time Password (OTP)** from the **app**
  → ***Valid for 12 hours***

- **Access to the cluster via ssh**

```
$ ssh <username>@login.<cluster>.cineca.it
```

https://docs.hpc.cineca.it/general/access.html

# Access via 2-factor authentication

## "Any" access to the cluster

- **Request the ssh certificate** to our Identity Provider via the smallstep client
  → A web page will open on the **browser** and you will be asked to insert **HPC username and password**, and a **One-Time Password (OTP)** from the **app**
  → ***Valid for 12 hours***

- **Access to the cluster via ssh**

```
$ ssh <username>@login.<cluster>.cineca.it
```

https://docs.hpc.cineca.it/general/access.html

# Project Account

The Project Account identifies the resource allocation that you can use for submitting jobs.

- A **budget** is associated with an account and reports how many **computing hours** (**cpus-h**) are available on the cluster. When you run jobs on **compute nodes**, you consume this budget.
- An amount of **storage** is also associated with a Project Account, available on the **$WORK** and **$FAST** areas.

- The account budget and storage are shared between all the users that are associated to the account (***PI and collaborators***).
- A single username can use **multiple accounts,** and an account can be used by **multiple usernames**, all competing for the same budget.
- When the account runs **out of budget**, or when it reaches the **expiring date**, all the usernames referring to that account won't be able to submit jobs anymore.

# Filesystems

## $HOME

- 50 GB per user
- user specific
- permanent
- daily backup (soon)

## $PUBLIC

- 50 GB per user
- user specific (permissions **755**)
- permanent
- **no** backup

## $SCRATCH

- no quota
- user specific
- temporary (data removed after 40 days)
- **no** backup

# Filesystems

## $HOME

- 50 GB per user
- user specific
- permanent
- daily backup (soon)

## $PUBLIC

- 50 GB per user
- user specific (permissions **755**)
- permanent
- **no** backup

## $SCRATCH

- no quota
- user specific
- temporary (data removed after 40 days)
- **no** backup

## $WORK

- quota per account (default 1 TB)
- account specific
- permanent
- **no** backup

## $FAST

- like $WORK
- **fast I/O**
- only on Leonardo

# Filesystems

### $HOME
- 50 GB per user
- user specific
- permanent
- daily backup (soon)

### $PUBLIC
- 50 GB per user
- user specific (permissions **755**)
- permanent
- **no** backup

### $SCRATCH
- no quota
- user specific
- temporary (data removed after 40 days)
- **no** backup

### $WORK
- quota per account (default 1 TB)
- account specific
- permanent
- **no** backup

### $FAST
- like $WORK
- **fast I/O**
- only on Leonardo

### $TMPDIR
- local on nodes
- job specific

### DRES
- long storage on demand
- shared among accounts and platforms (not Leonardo)

All the filesystems are based on **Lustre**

→ Check your areas, disk usage and quota:

**$ cindata**

**$ cinQuota**

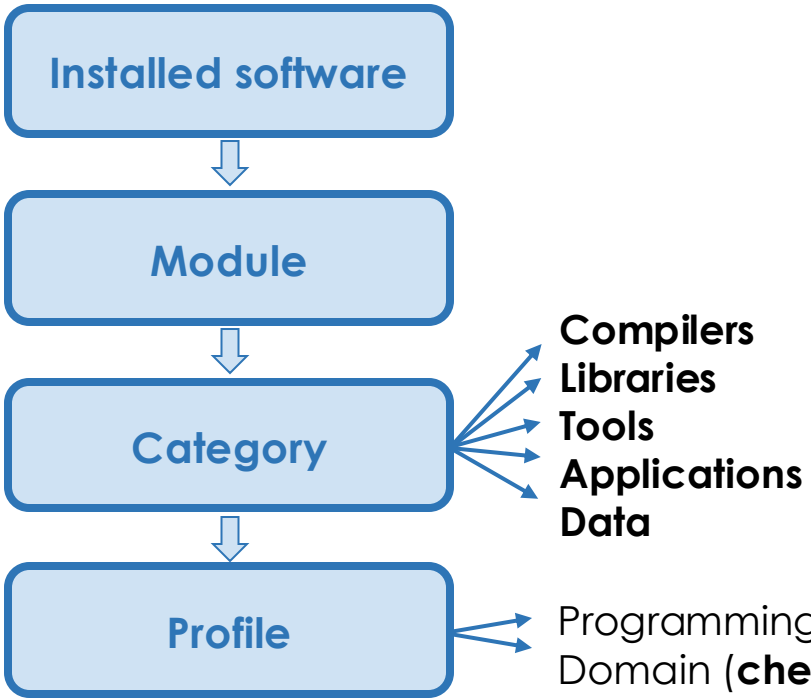
https://docs.hpc.cineca.it/hpc/hpc_data_storage.html

# OUTLINE

- Leonardo infrastructure
- Access HPC resources and filesystems
- **Software environment**
- Programming environment
- Production environment
- Final remarks

# Module environment

Any **available software** is offered on the clusters in a module environment.
The modules are organized in functional categories and collected in different profiles.




**Base** is the default profile:
automatically loaded after login,
containing basic modules
for programming activities

Programming (**base**, **advanced**): compilation, profiling, debugging
Domain (**chem-phys, astro, bioinf…**): production activities

# Module environment

**$ module av**

```
------------------------------------------------- /leonardo/prod/opt/modulefiles/profiles -------------------------------------------------
profile/archive  profile/base       profile/chem-phys  profile/geo-inquire  profile/meteo    profile/spoke7
profile/astro    profile/candidate  profile/deeplrn    profile/lifesc       profile/quantum  profile/statistics
```

```
------------------------------------------------- /leonardo/prod/opt/modulefiles/base/libraries -------------------------------------------------
adios/1.13.1--intel-oneapi-mpi--2021.10.0--oneapi--2023.2.0          metis/5.1.0--gcc--12.2.0
adios/1.13.1--openmpi--4.1.6--gcc--12.2.0-cuda-12.1                  metis/5.1.0--oneapi--2023.2.0
blitz/1.0.2--gcc--12.2.0                                             nccl/2.19.1-1--gcc--12.2.0-cuda-12.1
blitz/1.0.2--oneapi--2023.2.0                                        nccl/2.19.3-1--gcc--12.2.0-cuda-12.1
boost/1.83.0--gcc--12.2.0                                            netcdf-c/4.9.2--gcc--12.2.0
boost/1.83.0--intel-oneapi-mpi--2021.10.0--oneapi--2023.2.0-atomic   netcdf-c/4.9.2--intel-oneapi-mpi--2021.10.0--oneapi--2023.2.0
boost/1.83.0--oneapi--2023.2.0                                       netcdf-c/4.9.2--oneapi--2023.2.0
boost/1.83.0--openmpi--4.1.6--gcc--12.2.0                            netcdf-c/4.9.2--openmpi--4.1.6--gcc--12.2.0
boost/1.83.0--openmpi--4.1.6--nvhpc--23.11                           netcdf-c/4.9.2--openmpi--4.1.6--nvhpc--23.11
cfitsio/4.3.0--gcc--12.2.0                                           netcdf-fortran/4.6.1--gcc--12.2.0
```

```
------------------------------------------------- /leonardo/prod/opt/modulefiles/base/tools -------------------------------------------------
anaconda3/2023.09-0  jube/2.4.3          spack/0.21.0-68a
cintools/1.0         maven/3.8.4         spack/DCGP_0.21.0
```

```
------------------------------------------------- /leonardo/prod/opt/modulefiles/base/compilers -------------------------------------------------
cuda/12.1  gcc/12.2.0           intel-oneapi-compilers/2023.2.1     nvhpc/23.11  perl/5.36.0--gcc--8.5.0  python/3.10.8--gcc--8.5.0
cuda/12.3  gcc/12.2.0-cuda-12.1 llvm/14.0.6--gcc--12.2.0-cuda-12.1  nvhpc/24.3   perl/5.38.0--gcc--8.5.0  python/3.11.6--gcc--8.5.0
```

Almost all the modules on Leonardo have been installed with **Spack**, and they report the Spack package name.

# Module environment

**$ module load profile/astro**
**$ module av**

Loaded profiles are **added** to the environment

```
------------------------------------------------ /leonardo/prod/opt/modulefiles/profiles -------------------------------------------------
profile/archive  profile/base       profile/chem-phys  profile/geo-inquire  profile/meteo    profile/spoke7
profile/astro    profile/candidate  profile/deeplrn    profile/lifesc       profile/quantum  profile/statistics

------------------------------------------------ /leonardo/prod/opt/modulefiles/astro/libraries ------------------------------------------
cfitsio/4.3.0--gcc--12.2.0
```

**$ module show <module_name>/<version>** → Print information about the module, such as dependencies, paths

**$ module help <module_name>/<version>** → Print the help of the software, its brief description and examples of the use

https://docs.hpc.cineca.it/hpc/hpc_enviroment.html

# Module environment

**$ modmap -m <module_name>** → Detect all profiles, categories and modules available (e.g. different releases)

**$ module load <profile>**
**$ module load <module_name>/<version>** → all the dependencies are automatically loaded

**$ module list** → List all the profiles and modules loaded so far

*Important!*

You will find **modules compiled to support GPUs and modules suitable only for CPUs**.
You can check the compiler in the full name of the module, where the version is specified (e.g. gromacs/2022.3--intel-oneapi-mpi--2021.10.0--oneapi–2023.2.0). Remind that modules compiled with **gcc**, **nvhpc**, **cuda** should be used only on the **Booster partition**, while modules compiled with **intel oneapi** are suitable for running on the **DCGP partition**.

https://docs.hpc.cineca.it/hpc/hpc_enviroment.html

# Install new software

In case you don't find a software, you can choose to install it by yourself.

- **Install without sudo permissions**
- **Install with conda/pip in conda/virtual env**

  **Note:** the official conda repository is no more reachable from CINECA clusters. You can install Miniconda and rely on conda-forge repository: https://docs.hpc.cineca.it/services/miniconda.html
- **Install with Spack**

Write to superc@cineca.it if you need help on the installation or if you would like to request a new module.

# Install with Spack

A Spack environment is provided by the **package manager Spack** and available as **module**.

**$ module load spack/<version>**

- **setup-env.sh** file is sourced
- **$SPACK_ROOT** is initialized
- **spack command** is added to your PATH, and some nice command line integration tools as well
- Folder **/spack-<version>** is created into your **$PUBLIC** area (on Leonardo, and $WORK on the other clusters) and it contains some subfolders created and used by spack during the phase of the packages installation:
  - sources cache: **/cache**
  - software installation root: **/install**
  - modulefiles location: **/modules**
  - user scope: **/user_cache**

https://docs.hpc.cineca.it/hpc/hpc_enviroment.html#spack

# Install with Spack

## Some fundamental Spack commands

**$ spack list <package_name>** → Check if the package is available for installation with Spack

**$ spack info <package_name>** → Show available versions, building variants and dependencies

**$ spack spec -Il <package_name>** → Show version, compiler, dependencies, building variants with which the package will be installed (-Il for installation status and *hash*)
→ options can be specified

e.g. $ spack spec -Il scorep

```
Concretized
--------------------------------
-   ijz2tvy  scorep@7.1%gcc@11.3.0+mpi+papi~pdt~shmem~unwind build_system=autotools arch=linux-rhel8-icelake
-   5bnn3tq      ^cubelib@4.6%gcc@11.3.0 build_system=autotools arch=linux-rhel8-icelake
```

**$ spack install <package_name>** → Install the package
→ options as spec command

**$ spack load <package_name>** → Load the package installed to use it (you can also create a **module**)

# OUTLINE

- Leonardo infrastructure
- Access HPC resources and filesystems
- Software environment
- **Programming environment**
- Production environment
- Final remarks

# Programming environment

Compilers and MPI libraries are available as modules in profile/base.
**Use the ones suitable for the architecture!**

Check with commands
modmap -m,
module av,
module show,
module help,
and **man**

## Compilers

- **GCC** (GNU compilers: gcc, g++, gfortran)
- **NVHPC** (ex hpc-sdk, ex PGI + CUDA → NVIDIA compilers: nvc, nvc++, nvcc, nvfortran)
- **CUDA**
- **INTEL ONEAPI** (Intel compilers: icc, icpc, ifort. Oneapi compilers: icx, icpx, ifx) → **no** Nvidia GPU support

## MPI libraries

- **OpenMPI** (GNU/NVHPC compilers)
- **Intel Oneapi MPI** (Intel compilers) → **no** CUDA-aware

https://docs.hpc.cineca.it/hpc/hpc_enviroment.html#compilers

# OUTLINE

- Leonardo infrastructure
- Access HPC resources and filesystems
- Software environment
- Programming environment
- **Production environment**
- Final remarks

# Login and compute nodes

CINECA HPC clusters are shared among many users, so **a responsible use is crucial!**

## Login nodes

- Interactive runs on login nodes are strongly discouraged and should be limited to short test runs → ***10 minutes cpu-time limit***
- Avoid running large and parallel applications on login nodes
- ***No GPUs on login nodes***

## Compute nodes

- Long production jobs should be submitted on compute nodes using the **scheduler** → **SLURM**
- Jobs can be submitted in two main ways: via **batch mode** and via **interactive mode**
- **Nodes shared**, but the allocated resources (cores, GPUs, RAM, $TMPDIR) are assigned in an exclusive way

# Submit jobs with SLURM

## Batch mode

- Write a **batch script** and ask for the needed resources with **SLURM directives** (**#SBATCH**)
- **Launch the batch script**

**$ sbatch jobscript.sh**

- The job is queued and scheduled

```
#!/bin/bash

#SBATCH --nodes=1                       # nodes
#SBATCH --ntasks-per-node=4             # tasks per node
#SBATCH --cpus-per-task=8               # cores per task
#SBATCH --gres=gpu:4                    # GPUs per node
#SBATCH --mem=494000                    # mem per node (MB)
#SBATCH --time=00:30:00                 # time limit (d-hh:mm:ss)
#SBATCH --account=EUHPC_TDEMO_26        # project account
#SBATCH --partition=boost_usr_prod      # partition name
#SBATCH --qos=boost_qos_dbg             # quality of service
#SBATCH --exclusive                     # node in exclusive way
#SBATCH --out=%j.out                    # output file
#SBATCH --err=%j.err                    # error file

Export environment variables
Load modules, activate virtual/conda environment
Launch your application with srun/mpirun/python...
```

# Submit jobs with SLURM
## Interactive mode

- Ask for the needed resources with the same **SLURM directives** with **srun** or **salloc**
- The job is queued and scheduled
- **Launch your application** from that prompt: the standard input, output, and error streams are connected to the **terminal session**
- Exit from the terminal session: **$ exit**

**Non MPI programs** (one process using one or more GPUs)

```
$ srun -N 1 --ntasks-per-node=8  --cpus-per-task=4 --gres=gpu:4
-t 00:30:00 -p boost_usr_prod -q boost_qos_dbg
-A tra25_astrophd --pty /bin/bash
```

The session starts on the **compute node**: **[username@lrdn0053 ~]$**

Also **MPI programs** (using one or more GPUs)

```
$ salloc -N 1 --ntasks-per-node=8  --cpus-per-task=4 --
gres=gpu:4 -t 00:30:00 -p boost_usr_prod -q boost_qos_dbg
-A tra25_astrophd
```

A new session starts on the **login node**: **[username@login14 ~]$**

# Submit jobs with SLURM

## Project Account

**#SBATCH --account=EUHPC_TDEMO_26** or **-A EUHPC_TDEMO_26**

Specifies the account with a **budget** of core-hours available to run jobs.
As a user, you have access to a limited number of core-hours to spend. They are not assigned to User Accounts, but to **Project Accounts**, and are shared among users on the same project (i.e. your research partners).

On Leonardo, you can check the status of your accounts with

**$ saldo -b** → **Leonardo Booster**

**$ saldo -b --dcgp** → **Leonardo DCGP**

Accounts defined on Booster can only be used on **Booster partition** (boost_usr_prod), and accounts defined on DCGP can only be used on **DCGP partition** (dcgp_usr_prod).

**Your Project Account will be active until Sunday, March 15th.**

# Submit jobs with SLURM

## Partition and qos

**#SBATCH --partition=boost_usr_prod** or **-p boost_usr_prod**
Specifies the partition, i.e. the specific set of nodes among which your job can search for resources.

**#SBATCH --qos=boost_qos_dbg** or **-q boost_qos_dbg**
The Quality of Service is used to modify the limits of a partition and its priority, or to access selected partitions. If not specified, default qos "normal" will apply.

| Partition | QOS | #Cores/#GPU per job | Walltime | Max Nodes/cores/GPUs/user | Priority |
|---|---|---|---|---|---|
| boost_usr_prod | boost_qos_dbg | 2 nodes | 00:30:00 | 2 nodes / 64 cores / 8 GPUs | 80 |

https://docs.hpc.cineca.it/hpc/leonardo.html#job-managing-and-slurm-partitions

# Submit jobs with SLURM
## TMPDIR

- Only on Leonardo "diskful" nodes, it's possible to increase the space of the **$TMPDIR** area.
  Remind that the area is **local to nodes**, and **job specific** (i.e. "temporary"): created at the begging of a job and deleted at its end, and accessible only by the user who launched the job.

  Specify the space on $TMPDIR=/tmp (**default=10GB**):

  **#SBATCH --gres=tmpfs:200GB**

  on the local disks on **lrd_all_serial** nodes (max 1 TB) and **dcgp_usr_prod** compute nodes (max 3 TB).
  **Remind that for the DCGP jobs the requested amount of gres=tmpfs resource contributes to the consumed budget, changing the number of accounted equivalent core hours.**

- It is possible to use the $TMPDIR=/scratch_local space also on the **login** nodes (14 TB shared among users, remove your files once they are no more necessary).

- On the diskless **boost_usr_prod** compute nodes, the $TMPDIR=/tmp area is hosted on the RAM, with a fixed size of 10 GB (no increase is allowed, and the gres=tmpfs resource is disabled).

https://docs.hpc.cineca.it/hpc/leonardo.html#file-systems-and-data-managment

# Resources per Booster compute node

***Accounted Resources (cpus-h) = Equivalent reserved cores (cpus) • Elapsed Time (h)***

**Each node** → max resources you can request per Booster compute node

- 32 cores (**cpus**) → **ntasks-per-node * cpus-per-tasks ≤ 32**
- 4 GPUs (**gres=gpus**)
- 494000 MB of RAM (**memory**)

The **accounting** considers
- the requested number of CPUs
- the requested number of GPUs
- the requested memory on RAM

and calculates the **number of equivalent cores** → it takes the **maximum** among
- number of CPUs
- number of GPUs * 8 ( = number of GPUs * cores-per-node / GPUs-per-node)
- memory / memory-per-core ( = requested memory / memory-per-node * cores-per-node )

# Resources per DCGP compute node

*Accounted Resources (cpus-h) = Equivalent reserved cores (cpus) • Elapsed Time (h)*

**Each node** → max resources you can request per DCGP compute node

- 112 cores (**cpus**) → **ntasks-per-node * cpus-per-tasks ≤ 112**
- 494000 MB of RAM (**memory**)
- 3 TB of temporary local memory on $TMPDIR (**gres=tmpfs**)

The **accounting** considers
- the requested number of CPUs
- the requested memory on RAM
- the requested memory on $TMPDIR

and calculates the **number of equivalent cores** → it takes the **maximum** among
- number of CPUs
- memory / memory-per-core ( = requested memory / memory-per-node * cores-per-node )
- tmpfs / tmpfs-per-core ( = requested tmpfs / tmpfs-per-node * cores-per-node )

# Budget linearization

Every account has a **monthly quota** (total_budget / total_no_of_months).

When users start to consume the account budget, the jobs submitted from the account will **gradually lose priority**, until the monthly budget is fully consumed.

When this happens, you can still run jobs, so **it is possible to consume more than the monthly quota each month**, but the submitted jobs will have the lowest priority on the queue on pending jobs.

# Monitor your jobs with SLURM

**$ squeue -u <username>** or **$ squeue --me**
Shows the list of all your scheduled jobs, along with their status (pending, running, closing, …).
Also, shows you the **jobID** required for other SLURM commands.

**$ scontrol show job <job_id>**
Provides a long list of informations for the job requested.
In particular, if your job isn't running yet, you'll be notified about the reason it has not started yet and, if it is scheduled with top priority, you will get an **estimated start time**.

**$ scancel <job_id>**
Removes the job (queued or running) from the scheduled job list by killing it.

**$ sinfo** (e.g. **$ sinfo -o "%10D %a %20F %P"**)
Provides information about SLURM nodes and partitions.

**$ sacct <options> <job_id>** (e.g. **$ sacct -Bj <job_id>**)
Displays accounting data for all jobs and job steps in the SLURM job accounting log or SLURM database.

# OUTLINE

- Leonardo infrastructure
- Access HPC resources and filesystems
- Software environment
- Programming environment
- Production environment
- **Final remarks**

# Final remarks

- **2FA method** is mandatory on CINECA HPC systems (not for training usernames).
- **Login nodes** should only be used for installation (connection to external network!), compilation, and small tests. **No** GPUs on login nodes!
- Consider to use **Leonardo Booster** **for your applications on GPUs** and **Leonardo DCGP** **for applications only on CPUs.**
- Recommended **compilers** are gcc and Nvidia compilers (nvhpc, cuda) for Leonardo Booster, and gcc and Intel (intel oneapi) for Leonardo DCGP.
  Check the **options** required to enable OpenACC/OpenMP parallelization, GPU support…
- Rely on the already available **software stack**, tested and optimized for the cluster architecture, and on **Spack** for autonomously installing additional software.

https://docs.hpc.cineca.it and write to superc@cineca.it in case of need!


# Thank you and enjoy the cluster!

**Caterina Caravita**
HPC Production and Support Specialist

c.caravita@cineca.it

February 18th, 2026